

Table of Contents

20 Bug Bounty Lab Practices — VulnCorp Helpdesk on Metasploitable2

A full-workflow lab manual: recon → mapping → exploitation → chaining → reporting, using VulnCorp Helpdesk deployed on Metasploitable2 (or via `setup-modern.sh` / Docker) as the live target. Each lab names its required tools, the difficulty tier to set, numbered steps, a success check, and a “Report it” note showing what that finding would look like in a real submission — so by Lab 20 you’re not just exploiting bugs, you’re practicing the full bounty-hunter workflow.

Before Lab 1: confirm the app is deployed and reachable (README.md section 1 or 2), and you know its IP/URL. Set difficulty via **Admin Panel** → **Difficulty Settings** as each lab specifies. Seeded accounts: admin/admin123, sam/support123 (support), alice/alice123, bob/bob123 (both user).

Phase A — Recon & Mapping (Labs 1-4)

Lab 1: Full Port & Service Enumeration

Tools: nmap **Tier:** any **Objective:** Build a complete picture of what’s running on the target before touching the web app at all.

Steps:

1. Ping-sweep the subnet to confirm the target is up: `nmap -sn <subnet>/24`
2. Run a full TCP port scan with version detection: `nmap -sV -p- <target-ip>`
3. Record every open port and its reported service/version in your notes (use the Recon Note Template from the Toolkit).
4. Cross-reference the results against known service versions — note anything with a publicly known CVE (e.g. vsftpd 2.3.4 on Metasploitable2, PHP version, Apache version).
5. Identify which port serves VulnCorp Helpdesk specifically.

Success check: You have a documented list of every open port, its service, and version, with the VulnCorp Helpdesk port identified.

Report it: Not a finding on its own, but this becomes your report’s “**Scope and Methodology**” section — real reports document what was tested, not just what was found.

Lab 2: Web Stack Fingerprinting

Tools: whatweb, curl, browser DevTools **Tier:** simple **Objective:** Identify the exact technology stack serving the app without being told.

Steps:

1. Run whatweb `http://<target>/vulnapp/` and record what it reports for server, language, and framework.
2. Manually check response headers: `curl -I http://<target>/vulnapp/index.php` — note Server and any X-Powered-By.
3. Trigger a deliberate error (e.g. submit a malformed request) and see if any stack trace or version info leaks.
4. Compare your findings to what section 1 of the README documents about the target stack — did you independently arrive at the same answer?

Success check: You can state the exact web server, PHP version (or version range), and database in use, backed by evidence you collected yourself.

Report it: If a verbose version header or stack trace leaked, that's a **Low/Informational** finding — "Server version disclosure via HTTP headers." Note it for your final report even though it's not the main finding.

Lab 3: Content & Endpoint Discovery

Tools: ffuf or gobuster or dirb, SecLists wordlists **Tier:** hard (this is where the payoff endpoint exists) **Objective:** Discover an endpoint that isn't linked from anywhere in the UI.

Steps:

1. Set difficulty to **hard** via Admin Panel.
2. Run `ffuf -u http://<target>/vulnapp/user/FUZZ.php -w /usr/share/seclists/Discovery/Web-Content/common.txt -mc 200,301,302,403`
3. Also fuzz the admin/, support/, and root directories separately.
4. For every hit, manually visit it and note what it does, what HTTP methods it accepts, and whether it enforces authentication.
5. Compare your discovered endpoint list against what's actually linked in the dashboard navigation — what did you find that isn't linked anywhere?

Success check: You've discovered `profile_export.php` (or an equivalent hidden endpoint) purely through brute-forcing, before reading any hint.

Report it: This discovery technique — "endpoint enumeration via content discovery" — belongs in your methodology section, and the endpoint itself likely feeds into Lab 12's finding.

Lab 4: Full Application Mapping with Burp Suite

Tools: Burp Suite (Proxy + Site Map) **Tier:** any **Objective:** Build a complete map of the app's pages, forms, parameters, and role boundaries.

Steps:

1. Configure your browser to proxy through Burp (127.0.0.1:8080).
2. Log in and click through the app as each of the four seeded accounts in turn (admin, sam, alice, bob).
3. In Burp's Site Map, review every distinct URL, form, and parameter discovered.
4. Note which pages return a 403 for which roles — this tells you where access control is *supposed* to apply.
5. Identify the session mechanism (cookie name, whether it's HttpOnly/Secure) and where the difficulty-tier setting lives (hint: it's server-side, not in a cookie or hidden field).

Success check: A written map (screenshot or list) covering all four roles' visible pages, plus your notes on where role-based restrictions appear to exist.

Report it: This map becomes the backbone of your final report's scope description, and directly informs which endpoints you'll test for access-control bugs in Labs 8, 12, and 17.

Phase B — Simple Tier: Finding the Obvious Bugs (Labs 5–9)

Lab 5: SQL Injection Authentication Bypass

Tools: Browser or Burp Repeater **Tier:** simple **Objective:** Log into the app as admin without knowing the password.

Steps:

1. Set difficulty to **simple**.
2. On the login form, enter a single quote (') in the username field and submit — observe the error behavior.
3. Reason about the underlying query structure based on the error.
4. Craft a payload that closes the string early and neutralizes the rest of the query: try `admin' - -` as the username, any value as password.
5. Confirm you're logged in as admin without ever knowing admin123.

Success check: You're on the admin dashboard having never entered the real password.

Report it: Critical. Title: *"SQL Injection in Login Form Allows Full Authentication Bypass."* Include the exact payload, a screenshot of the resulting authenticated session, and note the business impact: complete compromise of any account, including admin, with zero prior credentials.

Lab 6: Data Extraction via UNION-Based SQLi

Tools: Burp Suite, sqlmap **Tier:** simple **Objective:** Extract every username and password hash from the database through the login form alone.

Steps:

1. With difficulty still on **simple**, capture the login POST request in Burp and save it as a .req file.
2. Run `sqlmap -r login.req -p username --dump` and let it identify and exploit the injection automatically.
3. Separately, manually determine the column count via `UNION SELECT` trial and error, and craft a manual payload that reflects data back into the page.
4. Compare what sqlmap found against your manual extraction.

Success check: A full dump of the `users` table, including password hashes, obtained via at least one method.

Report it: Same finding as Lab 5, but now with concrete evidence of data exposure — attach the extracted table (redact real credentials if this were a live program) as your Proof of Concept. This upgrades the report's **Impact** section from “could log in as anyone” to “extracted the entire user database.”

Lab 7: Stored XSS via Support Tickets

Tools: Browser DevTools **Tier:** simple **Objective:** Get JavaScript to execute in another user's browser via a submitted support ticket.

Steps:

1. Log in as `alice`. Submit a ticket with `<script>alert(document.cookie)</script>` in the message field.
2. View “My Tickets” and confirm the script executes.
3. Think through the chain: support agents (`sam`) read every ticket in the queue. Log in as `sam` and view the Support Queue — does the same script fire there too?
4. Consider (don't need to build) what a real payload would do instead of `alert()` — e.g., exfiltrate the session cookie to an attacker-controlled listener.

Success check: Confirmed script execution both in your own ticket view and in the support agent's queue view.

Report it: Medium-High, depending on who it reaches. Title: “*Stored XSS in Ticket Submission Executes in Support Agent Context.*” Note explicitly that this isn't self-XSS — it reaches a *different, higher-privileged* user (the support agent) without any action from them beyond normal job duties.

Lab 8: IDOR — Viewing and Editing Other Users' Profiles

Tools: Browser **Tier:** simple **Objective:** View and modify another user's profile data using only your own low-privilege session.

Steps:

1. Log in as `alice`. Note your own profile URL and its `id` parameter.
2. Change the `id` value to another user's ID (try 1 through 4) and reload.
3. Confirm you can see full profile details for accounts that aren't yours — including the password hash field, visible at this tier.
4. Attempt to *edit* another user's profile (not just view) via the same page.

Success check: Full read and write access to at least one other account's profile data.

Report it: High. Title: *"Insecure Direct Object Reference in User Profile Allows Unauthorized Data Access and Modification."* Note both the read (info disclosure) and write (data integrity) impact separately — they're two distinct consequences of the same root cause.

Lab 9: Command Injection via Admin Diagnostics

Tools: Browser, Burp Suite **Tier:** simple **Objective:** Execute an arbitrary OS command on the server via the network diagnostics tool (admin access required).

Steps:

1. Log in as `admin`, navigate to the Network Diagnostics tool.
2. Submit a normal hostname (e.g. `127.0.0.1`) and confirm the ping output renders.
3. Append a shell metacharacter and a harmless command: `127.0.0.1; id`
4. Confirm the output includes the result of `id`, not just the ping.
5. Try a second, different separator (`&&`) to confirm the vulnerability isn't specific to one character.

Success check: Arbitrary command output (e.g. `uid=0 (root)` or similar) appearing in the tool's response.

Report it: Critical. Title: *"OS Command Injection in Network Diagnostics Tool Allows Remote Code Execution."* This is your strongest finding so far — full RCE. Include the exact payload and full output as PoC.

Phase C — Intermediate Tier: Filter Evasion (Labs 10–12)

Lab 10: Bypassing the SQLi Keyword Filter

Tools: Burp Suite **Tier:** intermediate **Objective:** The app now strips `union`, `select`, `--`, `#`, `;` (lowercase only). Bypass login anyway.

Steps:

1. Set difficulty to **intermediate**. Retry Lab 5's exact payload — confirm it now fails.
2. Test whether the filter is case-sensitive by submitting a single blocked keyword in mixed case.
3. Recall that the original auth-bypass payload doesn't strictly need any of the blocked keywords — only an always-true condition and a comment. Reconstruct a bypass using only unfiltered characters.
4. Document exactly why your new payload evades the specific filter logic (case-sensitivity, or keyword-independence).

Success check: Authenticated as admin again, this time past an active filter.

Report it: Update your Lab 5 report to note: *"The vendor's proposed mitigation (keyword blacklist) is insufficient — see attached bypass."* This is a realistic real-world scenario: reporting that a partial fix doesn't actually close the finding.

Lab 11: XSS Filter Bypass

Tools: Browser DevTools, PortSwigger XSS cheat sheet (public reference) **Tier:** intermediate

Objective: The app now strips `<script>` tags case-insensitively. Get JavaScript to execute anyway.

Steps:

1. Confirm your Lab 7 payload is now neutered.
2. Recall that a browser executes JavaScript from many contexts beyond `<script>` tags — event handler attributes fire on their triggering event regardless of tag name.
3. Submit a ticket containing `<img src=x onerror=alert(1)>` and confirm execution.
4. Try at least one more vector (e.g. an `<svg onload=...>` payload) to confirm the filter's blind spot is systemic, not a one-off.

Success check: Confirmed script execution using a payload containing no literal `<script>` substring.

Report it: Same update pattern as Lab 10 — note in your report that tag-name-based blacklisting doesn't address the underlying lack of output encoding.

Lab 12: File Upload Filter Bypass (Content-Type Spoofing)

Tools: Burp Suite (Repeater) **Tier:** intermediate **Objective:** The upload feature now checks the file's Content-Type header. Bypass it to upload a PHP file.

Steps:

1. Attempt to upload a .php file normally through the Avatar Upload feature — confirm it's rejected.
2. Intercept the upload request in Burp before it reaches the server.
3. Edit only the Content-Type field in the multipart body for your file part to image/jpeg (or similar), leaving the actual file bytes and filename untouched.
4. Forward the request and confirm the upload now succeeds.
5. Browse directly to the uploaded file's URL and confirm your PHP code executes.

Success check: A .php file uploaded and executing, despite the Content-Type check.

Report it: Critical (same class as any unrestricted-upload RCE). Note specifically in your report *which* control was bypassed and *how* — “validation trusts a client-supplied header instead of inspecting file content” is the root-cause line a developer needs to fix it correctly.

Phase D — Hard Tier: Chained & Secondary Flaws (Labs 13–16)

Lab 13: Cookie-Based SQL Injection via “Remember Me”

Tools: Burp Suite **Tier:** hard **Objective:** The login form itself is now fully parameterized. Find the SQLi that moved elsewhere in the auth flow.

Steps:

1. Set difficulty to **hard**. Confirm the login form itself resists every prior SQLi payload.
2. Log in normally with “Remember me” checked; inspect the resulting cookie.
3. Reason about where else in the app a stored value like this might get read back into a query — a remember-me feature has to look up the token somewhere.
4. Log out, then manually set a crafted remember_token cookie value designed to manipulate that lookup query, and reload the login page.
5. Confirm you're auto-logged-in as an arbitrary account without a username/password at all.

Success check: Authenticated session established purely via a forged cookie value.

Report it: Critical. Title: “SQL Injection via Forgeable ‘Remember Me’ Cookie Bypasses Authentication Entirely.” Emphasize in your report that fixing the *primary* injection point doesn't guarantee the vulnerability class is gone from the codebase — this is a strong point to make to a client who thinks one fix “solved SQLi.”

Lab 14: Broken Access Control — The Forgotten Endpoint

Tools: Burp Suite, ffuf (from Lab 3) **Tier:** hard **Objective:** Direct profile access is now correctly restricted to your own account. Find where that fix wasn't applied.

Steps:

1. Confirm Lab 8's IDOR now correctly 403s at this tier.
2. Revisit your Lab 3 endpoint-discovery notes, or re-run the fuzzing if you skipped it — look specifically for an “export,” “api,” or “download” style endpoint.
3. Test the same IDOR technique (changing the target ID) against that endpoint instead of the main profile page.
4. Confirm it returns full profile data for arbitrary user IDs, unauthenticated by role.

Success check: Data exposure confirmed on the secondary endpoint even though the primary one is fixed.

Report it: High. Title: “Broken Access Control on `/user/profile_export.php` Bypasses Fixed IDOR Protection.” This is a great example of a **defense-in-depth** finding for your report: the fix was applied to one code path but not a functionally-identical one — flag this as a systemic code-review gap, not just a single bug.

Lab 15: Polyglot File Upload for RCE

Tools: exiftool or a hex editor **Tier:** hard **Objective:** The app now validates real image structure via `getimagesize()`. Get PHP execution anyway.

Steps:

1. Confirm a plain `.php` file is now rejected by the image-structure check.
2. Build a polyglot: prepend the 6-byte GIF header (GIF89a) to a one-line PHP payload (`<?php echo "pwned"; ?>`).
3. Verify locally that `getimagesize()` (or an online equivalent check) accepts this file as a valid GIF.
4. Upload the file with a `.php` extension via the Avatar Upload feature.
5. Browse to the uploaded file and confirm the PHP portion executes (look for your echoed string in the output, not just the raw file).

Success check: Confirmed PHP execution from a file that also independently validates as a real image.

Report it: Critical. Title: “Image-Validation Bypass via Polyglot File Enables Remote Code Execution.” Be precise in your report about *why* this works: `getimagesize()` confirms a file *starts with* valid image data, not that it contains *only* image data — a subtle but important distinction for the fix recommendation.

Lab 16: Race Condition — Multi-Claiming a One-Time Bonus

Tools: Burp Suite Intruder (concurrent mode) or a short async script (Python + aiohttp) **Tier:** simple first, then intermediate **Objective:** Claim a one-time “welcome bonus” more than once by exploiting a check-then-write timing gap.

Steps:

1. Set difficulty to **simple**. Claim the bonus once normally; confirm your balance is 100 credits.
2. Reset your account (or use a fresh one), then fire 10-15 genuinely concurrent requests at the claim endpoint (multiple browser tabs clicking at once, or several parallel `curl` commands launched together).
3. Check your final balance — more than 100 confirms multiple successful claims.
4. Switch to **intermediate** and repeat — note that a handful of manually-fired requests now mostly fails, because the timing window is much narrower.
5. Use a tool built for real concurrency (Burp Intruder's concurrent request mode, or a short async script) to land the intermediate-tier race reliably.

Success check: A final balance greater than 100 credits at both tiers, using different techniques for each.

Report it: High. Title: “*Race Condition in Bonus Claim Allows Unlimited Multi-Claiming.*” Note in your report which tooling was required — this matters for severity discussions, since “exploitable with a browser” and “exploitable only with specialized tooling” are meaningfully different risk profiles even for the same root cause.

Phase E — Expert Tier: Logic Flaws & Confirming Fixes (Labs 17–18)

Lab 17: Privilege Escalation via Mass Assignment

Tools: Burp Suite (Repeater) **Tier:** expert **Objective:** The profile-edit form shown to regular users has no role field. Become admin anyway.

Steps:

1. Set difficulty to **expert**. Log in as `alice`, submit a normal profile update, and capture the POST request in Burp.
2. Reason about how the server-side handler likely processes the submission — does it only save fields the form displayed, or everything received?
3. Add a parameter the visible form never included: `role=admin`.
4. Resend the modified request and confirm success.
5. Log out and back in as `alice` — confirm your role has actually changed.

Success check: `alice`'s account now has the `admin` role, achieved purely by adding an unexpected POST field.

Report it: Critical. Title: “*Mass Assignment in Profile Update Allows Privilege Escalation to Administrator.*” This is a good report to practice writing precisely: the vulnerability isn't in what the UI shows, it's in what the server accepts — make that distinction explicit for the developer reading your report.

Lab 18: Confirming a Fix — CSRF Token Validation

Tools: Browser, a scratch HTML file **Tier:** hard, then expert **Objective:** Build a CSRF proof-of-concept against an unprotected endpoint, then confirm the same PoC correctly fails once a token is required.

Steps:

1. Set difficulty to **hard**. Build a minimal auto-submitting HTML page targeting `/admin/create_user.php` with fields for a new backdoor admin account (username, password, role=admin, etc.).
2. While logged in as admin in one browser tab, open your crafted page in another tab.
3. Log out, then log in with the credentials you set — confirm a persistent backdoor admin account now exists, created without the real admin ever intending to.
4. Switch difficulty to **expert** and reload/resubmit the exact same PoC page.
5. Confirm it now fails, and capture the specific error message returned.

Success check: Successful backdoor creation at hard tier; a clean, specific rejection at expert tier.

Report it: This is two report entries in one lab. First, **Critical:** “*CSRF on Admin User Creation Allows Silent Backdoor Account Creation.*” Second, a **remediation verification note:** confirm to the client that the CSRF token fix works as intended, including your re-test evidence — real bounty programs often want confirmation that a patch actually closes the reported issue, not just the original finding.

Phase F — API Testing & Full Report Writing (Labs 19–20)

Lab 19: API Security Testing — Broken Object Level Authorization

Tools: Postman or Burp Suite, curl **Tier:** simple through expert (test all four) **Objective:** Practice testing a JSON API specifically, not just HTML forms, against `/api/tickets.php`.

Steps:

1. Set difficulty to **simple**. With no authentication at all, request `/api/tickets.php` and `/api/tickets.php?id=1` — confirm full, unauthenticated data exposure.
2. Set difficulty to **intermediate**. Log in as `alice` and request another user’s ticket by ID — confirm the API requires login but performs no ownership check.
3. Set difficulty to **hard**. Confirm the single-ticket lookup is now correctly scoped, then test the *list* mode (no `id` parameter) — confirm it still leaks every ticket, since the fix wasn’t applied to that code path.
4. Set difficulty to **expert**. Confirm both modes are now correctly scoped to the authenticated user.
5. Throughout, use Postman (or Burp) to save each request as a reusable collection — this is standard practice for API testing on real engagements, where you’ll re-run the same requests against many parameter values.

Success check: A documented set of requests/responses showing the exact access-control behavior at each of the four tiers.

Report it: **High** for the simple/intermediate findings, note the hard-tier list-mode gap as its own distinct finding (“**BOLA in List Mode Persists After Single-Object Fix**”) — same lesson as Lab 14, now in an API context. This is a good moment to practice writing a report for an API finding specifically: no screenshots of a UI, just clean request/response pairs as your PoC.

Lab 20: Full Engagement Report — Compiling Labs 1–19

Tools: The Report Writing Template and Severity Cheat Sheet (Toolkit) **Tier:** N/A — this is the writing lab **Objective:** Compile your strongest 5–8 findings from Labs 1–19 into a single, professional engagement report, as if VulnCorp Helpdesk were a real program you were submitting to.

Steps:

1. Review your notes from every prior lab and select the 5–8 most significant, well-evidenced findings — prioritize variety (don’t submit five SQLi findings when you also have RCE, IDOR, and a race condition available).
2. For each finding, use the **Report Writing Template**: a specific title, severity (rated via the **Severity Cheat Sheet**, not guessed), exact steps to reproduce, a clean PoC (request/response or screenshot), business-level impact, and a remediation recommendation specific enough for a developer to act on.
3. Write an **Executive Summary** at the top: 3–4 sentences, no jargon, stating the overall risk posture — this is what a non-technical stakeholder reads first.
4. Order your findings by severity, Critical first.
5. For at least one finding, explicitly note a **chain**: e.g., “the stored XSS in Lab 7, combined with the CSRF from Lab 18, could let an unauthenticated attacker gain full admin access with zero valid credentials at any point” — chaining lower-severity bugs into a higher-impact story is a core bounty-hunting skill, not just listing bugs in isolation.
6. Proofread for one thing specifically: could a developer who has never seen this app reproduce every finding using only what you wrote? If not, add detail until they could.

Success check: A complete, submission-ready report covering at least 5 distinct vulnerability classes, correctly severity-rated, with at least one explicit vulnerability chain called out.

Report it: This is the report. Treat it as your portfolio piece — this is the artifact you’d actually want to show a real bug bounty program, a hiring manager, or use as a writing sample when applying to programs that ask for one.

After Lab 20

You’ve now run the full workflow this app was built to teach: recon, mapping, straightforward exploitation, filter evasion, chained/secondary flaws, logic bugs, API testing, and — critically —

turning all of it into a report a real program would actually accept. From here, the honest next step (per this app's own README) is DVWA and PortSwigger's Web Security Academy for the vulnerability categories a single custom app can't cover (JWT, GraphQL, HTTP smuggling, and more) — you now have the workflow habits to get real value out of them faster.